Farmacia Hospitalaria

Resumen de la carpeta de estudio

Sistema de Gestión de Stock — Farmacia Hospitalaria

Los 18 capítulos de la carpeta de estudio condensados para repasar: conceptos, código clave, reglas de negocio, glosario y preguntas de práctica.

Trabajo Final Integrador — Tecnicatura Universitaria en Programación — UTN
Noviembre de 2026

Índice

1. El proyecto en un vistazo 3
2. Repaso de programación 5
3. Cómo funciona la web 6
4. Bases de datos y SQL 8
5. Java 11
6. Spring Boot 14
7. Recorrido del backend 16
8. Seguridad 18
9. Reglas de negocio: FEFO, stock y concurrencia 21
10. JavaScript y TypeScript 25
11. React 26
12. Recorrido del frontend 29
13. Docker y entornos 31
14. Pruebas 33
15. Git y herramientas 35
16. Glosario rápido 36
17. Preguntas para practicar 37
18. Checklist antes de la defensa 39

Resumen de la carpeta de estudio

Este resumen junta lo esencial de los 18 capítulos de la carpeta de estudio, pensado para **repasar**. Cada sección explica la idea, muestra dónde aparece en el proyecto y cierra con lo que conviene saber decir en la defensa. Al final hay un glosario corto y preguntas con respuesta para practicar en voz alta.

Cómo usarlo: leé una sección, cerrá el PDF e intentá explicarla con tus palabras. Si te trabás, volvé a leer solo esa parte. Para el código completo de algún tema, está la carpeta de estudio.

> **Para la defensa (la presentación en un minuto):** "Es un sistema web de gestión de stock por lote para la farmacia de un hospital público. Registra ingresos y transferencias a las áreas aplicando FEFO automáticamente, avisa vencimientos y faltantes, deja trazabilidad de cada movimiento y genera reportes. Está hecho con un backend en Java con Spring Boot, una base PostgreSQL normalizada y un frontend en React, con seguridad por roles y pruebas automáticas."

1. El proyecto en un vistazo

El problema

Un hospital público tiene una **farmacia** con un depósito de medicamentos. Todos los días pasan dos cosas:

1. **Entra mercadería**: una droguería (proveedor) trae cajas. Cada caja dice un **número de lote** (por ejemplo `L2345`) y una **fecha de vencimiento**.
2. **Sale mercadería**: las áreas del hospital (Guardia, Terapia Intensiva, Pediatría…) piden medicamentos y la farmacia se los entrega.

Si no se controla bien, los medicamentos **se vencen** en el estante (plata tirada) o **faltan** cuando se necesitan (un problema para un paciente). Y si un laboratorio avisa "retiren el lote L2345", hay que saber **dónde está cada unidad** de ese lote.

Qué hace el sistema

- **Registra lo que entra**, lote por lote, con su vencimiento.
- **Registra lo que sale** a cada área. El sistema elige solo de qué lote sacar: **del que vence primero** (FEFO).
- **Avisa** qué está vencido, qué está por vencer y qué se está terminando.
- **Da de baja** lo vencido y **corrige** diferencias de inventario (ajustes).
- **Saca reportes** en Excel, CSV y PDF.
- Todo **con usuario y contraseña**, y cada persona hace solo lo que su rol le permite.

Las piezas (la analogía del restaurante)

Restaurante	Sistema	Tecnología
El salón y el menú	**Frontend**: las pantallas en el navegador	React 19 + TypeScript + Vite
El mozo que lleva los pedidos	**HTTP**: los mensajes entre navegador y servidor	JSON por HTTP
La cocina, con recetas y reglas	**Backend**: el servidor que aplica las reglas	Java 21 + Spring Boot 4.1
La despensa ordenada	**Base de datos**	PostgreSQL 16 + Flyway
El edificio	**Contenedores**	Docker



Regla de oro: el frontend **muestra** y el backend **decide**. Aunque el frontend esconda un botón, la seguridad y las reglas están en el backend, porque el frontend corre en la computadora del usuario y cualquiera podría modificarlo.

Roles

Rol	Quién es	Qué puede hacer
ADMIN	Jefe/a de farmacia	Todo, incluido crear usuarios.
FARMACEUTICO	Farmacéutico/a o técnico/a	Ingresos, transferencias, bajas, ajustes y editar el catálogo.
CONSULTA	Dirección, auditoría	Solo mirar y sacar reportes.

Las entidades (lo que guarda)

Entidad	Ejemplo	Datos principales
Usuario	`farmaceutico`	Nombre, contraseña (hash), rol, activo.
Proveedor	Droguería del Sur S.A.	Razón social, CUIT, teléfono, correo.
Área	Guardia	Nombre.
Medicamento	Amoxicilina, comprimidos 500 mg	Código, nombre, presentación, stock mínimo.
Lote	L2345, vence 30/11/2026, 200 unidades	Medicamento, proveedor, número, vencimiento, cantidad inicial y actual.
Transferencia	TR-000042 a Guardia	Fecha, área, quién la hizo, observaciones.
Detalle de transferencia	100 unidades del lote L2345	Qué lote y cuánto.
Movimiento de stock	−100 del lote L2345 por TR-000042	Cada cambio de cantidad: ingreso, egreso, baja o ajuste.
Recuperación de contraseña	Código para `consulta`	Código (hash), vencimiento, intentos.

Las 10 reglas para saber de memoria

1. **Vencido** = vence **antes de hoy**. El día del vencimiento todavía se usa.
2. **Por vencer** = vence entre hoy y dentro de 30 días.
3. **Stock total** de un medicamento = suma de sus lotes **no vencidos**. No se guarda: se calcula.
4. **Sin stock** = 0. **Stock bajo** = más de 0 pero ≤ stock mínimo.
5. **FEFO**: al transferir, sale primero el lote que vence primero.
6. **Todo o nada**: si falta stock de algún ítem, no se descuenta nada.
7. Un lote vencido **nunca** se transfiere.
8. **Nada se borra**: se desactiva (baja lógica).
9. El stock **nunca** queda negativo, ni siquiera si dos personas transfieren al mismo tiempo.
10. Cada cambio de stock deja un **movimiento** con quién, cuándo y por qué (**trazabilidad**).

Un día en el sistema

1. **8:00** — La farmacéutica ingresa. El **Panel** muestra 3 lotes vencidos, 2 medicamentos sin stock y 5 por vencer.
2. **8:05** — Da de baja los 3 lotes vencidos con el motivo "retirado para destrucción". Cada lote queda en 0 y se registra un movimiento de **baja**.
3. **9:30** — Llega Paracetamol. Registra un **ingreso**: lote L2350, vence en marzo, 500 unidades. Se crea el lote y un movimiento de **ingreso** (+500).
4. **10:00** — Guardia pide 150 Paracetamol. Hay un lote de 100 que vence en diciembre y el nuevo de 500 que vence en marzo. El sistema saca **100 del de diciembre y 50 del de marzo** (FEFO). Imprime el comprobante y lo firman.
5. **12:00** — Se rompieron 2 ampollas de Adrenalina. Hace un **ajuste** con el motivo "rotura" (−2).
6. **17:00** — La directora (CONSULTA) descarga el **reporte** de transferencias del mes en Excel.

2. Repaso de programación

Lo mínimo para leer el código. La lógica es la misma en Java (backend) y TypeScript (frontend); cambia la forma de escribir.

- **Compilar**: Java se compila a *bytecode* que ejecuta la **JVM**; TypeScript se compila a JavaScript que ejecuta el navegador. El compilador avisa los errores **antes** de ejecutar.
- **Variable**: caja con nombre. **Tipo**: qué cabe en la caja.

Idea	Java	TypeScript
Número entero	`int`, `long`, `Long` (puede ser null)	`number`
Texto	`String`	`string`
Verdadero/falso	`boolean`	`boolean`
Fecha sin hora	`LocalDate`	texto `'2026-11-30'`
Lista	`List<Lote>`	`Lote[]`
Diccionario	`Map<Long, Integer>`	`Record<string, number>`

> **Ojo:** `int` no puede ser `null`; `Integer` sí. Por eso en los datos de entrada se usa `Integer stockMinimo`: si no llega, vale `null` y `@NotNull` avisa. Y en Java los textos se comparan con `a.equals(b)`, nunca con `==`.

Bucles: el corazón del FEFO

```
for (Lote lote : lotes) {                // lotes ya ordenados por vencimiento
    if (restante == 0) break;
    int tomada = Math.min(restante, lote.getCantidadActual());
    lote.descontar(tomada);
    restante -= tomada;
}
```

Recorre los lotes y va tomando de cada uno hasta cubrir lo pedido. Es casi literal lo que hace `TransferenciaService.transferir`.

Clases, objetos y encapsulamiento

Una **clase** es un molde y un **objeto** es algo hecho con el molde. `Lote` es la clase; cada lote real (`L2345`) es un objeto.

```
public class Lote {
    private int cantidadActual;              // private: nadie de afuera lo toca

    public void descontar(int cantidad) {    // la única puerta, con la regla adentro
        if (cantidad > cantidadActual) {
            throw new IllegalStateException("No alcanza");
        }
        cantidadActual -= cantidad;
    }
}
```

Encapsulamiento: `cantidadActual` es privado y solo cambia con `descontar`, `ajustar` o `darDeBaja`, que controlan las reglas. A propósito, `Lote` **no tiene setters**. El **constructor** hace que el objeto nazca válido (rechaza una cantidad ≤ 0).

- **Herencia** (`extends`): `ManejadorErrores` aprovecha el manejo de errores que ya trae Spring.
- **Interfaz**: un contrato. `NotificadorRecuperacion` es la interfaz y `CorreoSimulado` la implementa; mañana se escribe `CorreoReal` sin tocar el resto.

Excepciones, colecciones y Optional

- Una **excepción** corta la ejecución y sube hasta que alguien la atrapa. En el proyecto **nadie las atrapa en el medio**: suben hasta `ManejadorErrores`, que las convierte en una respuesta HTTP.
- **Lista** (en orden), **conjunto** (sin repetidos, como los permisos de un rol) y **mapa** (clave → valor). Los ítems repetidos de una transferencia se suman con `cantidades.merge(id, cantidad, Integer::sum)`.
- `Optional`: "puede haber o no". `areaRepository.findById(id).orElseThrow(...)` = "buscá el área; si no está, 404". Nunca `Optional.get()` sin chequear.
- **Comentarios**: se comenta el **por qué**, no el **qué**. "El orden por id evita deadlocks" es útil; "suma uno a i" no.

3. Cómo funciona la web

Cliente, servidor y URL

El **cliente** (el navegador) pide y el **servidor** (el backend) responde, como en un mostrador. El cliente nunca entra a la cocina (la base): todo pasa por el mostrador.

```
http://localhost:8080/api/medicamentos/5?activo=true&page=0
protocolo  host  puerto     ruta              parámetros
```

Puertos del proyecto: backend **8080**, frontend de desarrollo **5173**, PostgreSQL **5432**.

HTTP

Cada pedido tiene **método**, **URL**, **cabeceras** y opcionalmente **cuerpo**. Cada respuesta tiene **código de estado**, cabeceras y cuerpo.

```
POST /api/areas HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{ "nombre": "Hemodiálisis" }
```

```
HTTP/1.1 201 Created
Location: /api/areas/9

{ "id": 9, "nombre": "Hemodiálisis", "activo": true }
```

Método	Para qué	Ejemplo
GET	Leer. No cambia nada.	`GET /api/medicamentos`
POST	Crear o ejecutar una acción.	`POST /api/transferencias`
PUT	Reemplazar los datos de algo.	`PUT /api/areas/3`
PATCH	Cambiar una parte.	`PATCH /api/areas/3/estado`
DELETE	Borrar.	**No se usa**: las bajas son lógicas.

Código	Nombre	Cuándo
200	OK	Consulta o edición correcta.
201	Created	Se creó algo (con `Location`).
202	Accepted	Pedido de código de recuperación aceptado.
204	No Content	Salió bien y no hay nada que devolver.
400	Bad Request	Datos inválidos.
401	Unauthorized	**"No sé quién sos"**: sin token, inválido o vencido.
403	Forbidden	**"Sé quién sos, pero no podés"**.
404	Not Found	No existe.
409	Conflict	Choca con una regla (stock insuficiente, código repetido).
500	Internal Server Error	Error inesperado del servidor.

HTTP no tiene memoria (*stateless*): cada pedido es independiente, por eso cada uno trae el token que dice quién sos.

JSON, API REST y paginación

- **JSON**: `{ }` es un objeto, `[ ]` una lista. Spring convierte Java ↔ JSON con **Jackson**.
- **API REST**: cada recurso tiene una URL (`/api/medicamentos/5`), se opera con los métodos HTTP, se responden los códigos correctos y es sin estado. Cada método + ruta es un **endpoint**.
- **Paginación**: no se mandan 2.000 movimientos juntos. `GET /api/movimientos?page=0&size=20&sort=fechaHora,desc` devuelve `content` (la página) y `page` (número, tamaño, total de elementos y de páginas).

Errores con formato estándar

```
{ "title": "Operación rechazada", "status": 409,
  "detail": "El área Guardia está inactiva.", "codigo": "AREA_INACTIVA" }
```

Es **Problem Details (RFC 9457)** más un `codigo` propio. `detail` es para la persona (el frontend lo muestra tal cual) y `codigo` es para el programa (ante `STOCK_INSUFICIENTE`, marca las líneas con faltante).

SPA, proxy y HTTPS

- **SPA** (*Single Page Application*): el navegador descarga el frontend una vez y después cambia de pantalla sin recargar; los datos se piden con `fetch`. Si recargás en `/medicamentos`, el servidor web devuelve igual `index.html` y React muestra la pantalla correcta.
- **Origen** = protocolo + host + puerto. El navegador no deja que una página lea respuestas de otro origen salvo que se autorice (**CORS**).
- **Proxy**: el navegador le pide todo a un solo origen y Vite (desarrollo) o nginx (producción) reenvían `/api` al backend. No hace falta CORS.
- **HTTPS** es HTTP cifrado. En el hospital se pone en el proxy, con el certificado del hospital.



4. Bases de datos y SQL

Tablas y claves

Una base **relacional** guarda los datos en **tablas**: cada fila es un registro y cada columna un campo con tipo fijo.

- **Clave primaria (PK)**: identifica cada fila, como un DNI. En todas las tablas es `id`, que la base asigna sola.
- **Clave foránea (FK)**: apunta a otra tabla. `lote.medicamento_id = 5` dice "este lote es del medicamento 5". La base no deja cargar un id que no exista (**integridad referencial**).
- **Sustituta vs. natural**: las PK son números sin significado (sustitutas); los datos únicos del negocio (CUIT, código) se protegen con `UNIQUE`. Así un código mal cargado se corrige sin romper relaciones.

Relaciones

Tipo	Ejemplo	Cómo se arma
Uno a muchos (1:N)	Un medicamento tiene muchos lotes.	FK del lado "muchos": `lote.medicamento_id`.
Muchos a muchos (N:M)	Una transferencia usa muchos lotes y un lote se reparte en muchas transferencias.	Tabla intermedia `detalle_transferencia`, con las dos FK y la cantidad.

Restricciones

Restricción	Qué hace	Ejemplo
`NOT NULL`	Obligatorio.	`nombre varchar(120) not null`
`UNIQUE`	No se repite.	`unique (codigo)`

Restricción	Qué hace	Ejemplo
`CHECK`	Condición que se cumple sí o sí.	`check (cantidad_actual >= 0 and cantidad_actual <= cantidad_inicial)`
`REFERENCES`	Tiene que existir en la otra tabla.	`references medicamento (id)`
`DEFAULT`	Valor si no se manda.	`activo boolean not null default true`

> **Para la defensa:** "El `CHECK` de `cantidad_actual >= 0` es la última barrera: aunque el código Java tuviera un error, la base rechaza un stock negativo."

SQL

```
-- Juntar tablas: el lote con el nombre del medicamento
select l.numero_lote, m.nombre, l.fecha_vencimiento, l.cantidad_actual
from lote l
join medicamento m on m.id = l.medicamento_id
where l.cantidad_actual > 0;

-- Agrupar y sumar: stock total por medicamento (sin vencidos)
select medicamento_id, sum(cantidad_actual) as stock, min(fecha_vencimiento) as proximo
from lote
where cantidad_actual > 0 and fecha_vencimiento >= :hoy
group by medicamento_id;
```

- `join` devuelve solo las filas con pareja; `left join` devuelve todas las de la izquierda (se usa para listar también los medicamentos sin lotes).
- La segunda consulta es casi la de `MedicamentoConsultasImpl`: el stock **se calcula** cada vez, con la fecha de hoy como parámetro.
- **Parámetros siempre**: pegar texto del usuario en el SQL permite **inyección SQL** (`x'; drop table usuario; --`). Con `:patron`, la base lo trata como dato. El `sort` no se puede parametrizar, así que se valida contra una **lista blanca** (`Ordenamiento.validar`).
- **Índices**: como el índice de un libro. Hay uno por `(medicamento_id, fecha_vencimiento)` y uno parcial solo para los lotes con stock.

Normalización

Normalizar es organizar las tablas **para no repetir datos**, porque los datos repetidos se terminan contradiciendo.

Forma	Regla	En el proyecto
1FN	Un valor por celda, sin columnas repetidas.	Los medicamentos de una transferencia son filas de `detalle_transferencia`.
2FN	Cada dato depende de la clave completa.	Todas las PK son un solo `id`: se cumple sola.
3FN	Ningún dato depende de otro que no sea la clave.	El nombre del área vive solo en `area`.

> **Truco:** cada dato depende "de la clave, de toda la clave y de nada más que la clave".

No se guarda	Porque depende de…	Se obtiene con…
Nombre del medicamento en el lote	`medicamento_id`	`JOIN`
Stock total del medicamento	sus lotes	`SUM`
Estado del lote	la fecha de hoy	se calcula
Número `TR-000042`	el `id`	se formatea
Total de unidades de la transferencia	sus detalles	`SUM`

Única excepción justificada: `movimiento_stock.saldo_resultante`. Es un dato **histórico** ("lo que había en ese momento") que nunca se modifica, así que no puede contradecirse. Sirve para auditoría.

Transacciones (ACID)

Una transacción agrupa operaciones que se hacen **todas o ninguna**. Si una transferencia falla en el segundo lote, se revierte (*rollback*) y todo queda como estaba.

Letra	Propiedad	Significa
A	Atomicidad	Todo o nada.
C	Consistencia	De un estado válido a otro válido (se respetan CHECK y FK).
I	Aislamiento	Las transacciones simultáneas no se pisan.
D	Durabilidad	Lo confirmado queda guardado aunque se corte la luz.

En el proyecto, cada método con `@Transactional` es una transacción: si lanza una excepción, Spring hace rollback. Por eso el "todo o nada" de las transferencias sale casi gratis.

Concurrencia, bloqueos y deadlock

Momento	Persona A	Persona B
1	Lee el lote: hay 100.	Lee el lote: hay 100.
2	Pide 60: alcanza.	Pide 60: alcanza.
3	Guarda 40.	Guarda 40.
Resultado	Se entregaron **120** de un lote de 100. ❌	

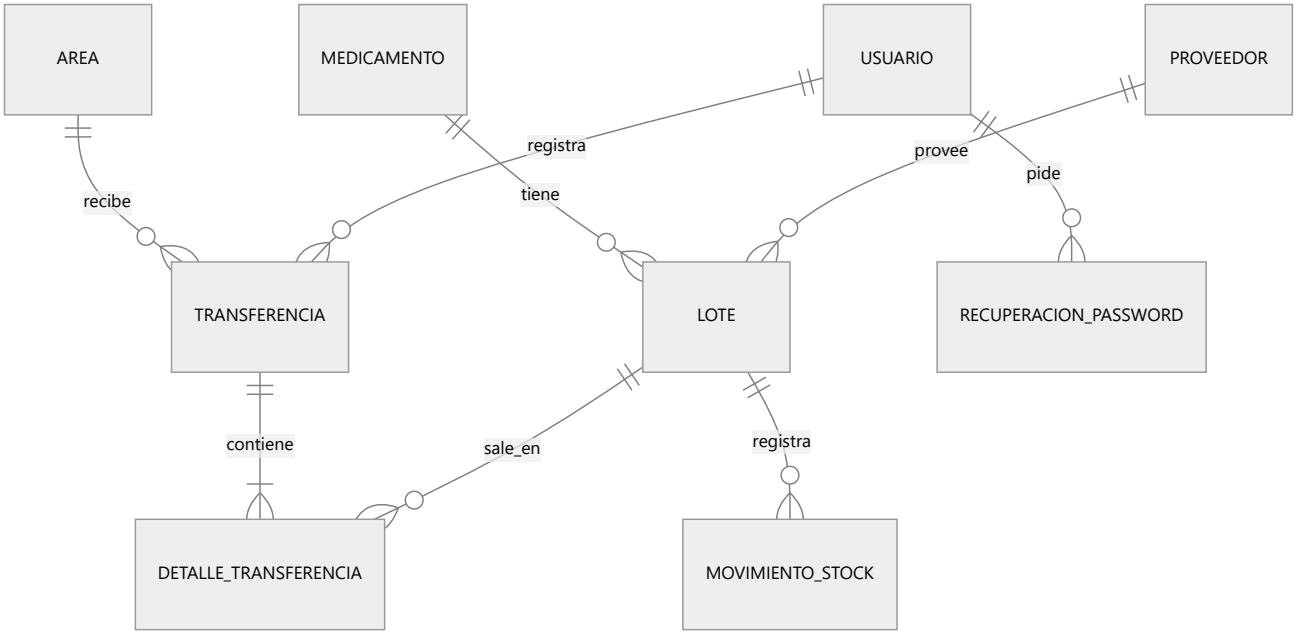
Eso es una **condición de carrera**. La solución es el **bloqueo pesimista**: `SELECT … FOR UPDATE` reserva las filas y la segunda persona **espera**; cuando lee, ve 40 y recibe un 409.

Deadlock: A bloquea Amoxicilina y quiere Paracetamol; B bloquea Paracetamol y quiere Amoxicilina. Se esperan para siempre. Se evita **bloqueando siempre en el mismo orden** (por id del lote).

Migraciones con Flyway y las 9 tablas

Flyway ejecuta en orden `V1__esquema.sql`, `V2__datos_base.sql`, `V3__recuperacion_password.sql`… y anota cuáles aplicó en `flyway_schema_history`. **Una migración aplicada nunca se edita**: para cambiar algo se crea una nueva. Hibernate está en `ddl-auto: validate`: solo verifica que las clases coincidan con las tablas.

Tabla	Guarda
`usuario`	Quién puede entrar y con qué rol.
`proveedor`	Quién nos vende.
`area`	A quién le entregamos.
`medicamento`	El catálogo.
`lote`	Cada partida recibida, con vencimiento y cantidad.
`transferencia`	Cada entrega a un área (cabecera).
`detalle_transferencia`	De qué lote salió cada parte de una entrega.
`movimiento_stock`	Cada cambio de cantidad de cada lote (auditoría).
`recuperacion_password`	Códigos para recuperar la contraseña.



5. Java

Java es el lenguaje del backend. Usamos **Java 21**, una versión de soporte extendido (LTS).

Estructura y tipos

```
package ar.edu.utn.farmacia.area;     // en qué paquete (carpeta) está
import jakarta.persistence.Entity;     // qué usa de otros paquetes

@Entity                                // anotación (etiqueta)
public class Area {                    // el archivo se llama Area.java
    private Long id;                   // atributos
    public Area(String nombre) { ... } // constructor
    public String getNombre() { ... }  // métodos
}
```

Los ids son `Long` (con mayúscula) porque un objeto recién creado todavía no tiene id: vale `null` hasta que la base lo asigna. Para dinero no se usa `double`.

Fechas y el reloj

Clase	Representa	Ejemplo
`LocalDate`	Un día sin hora.	Vencimiento `2026-11-30`.
`OffsetDateTime`	Un instante con su diferencia horaria.	`2026-11-09T17:49:00-03:00`
`Instant`	Un instante absoluto (UTC).	Vencimiento del token.
`Duration`	Una duración.	`8h` del token.
`Clock`	De dónde se saca la hora actual.	Con zona de Buenos Aires.

> **En el proyecto: nunca** `LocalDate.now()` sin el reloj. Siempre `LocalDate.now(clock)`. El servidor corre en UTC y a las 21:00 de Argentina ya es mañana. En las pruebas, el reloj es fijo.

Records: los DTO

```
public record AreaResponse(Long id, String nombre, boolean activo) {
    public static AreaResponse desde(Area area) {     // de entidad a respuesta
        return new AreaResponse(area.getId(), area.getNombre(), area.isActivo());
    }
}

public record MedicamentoRequest(String codigo, String nombre, String presentacion, Integer stockMinimo) {
    public MedicamentoRequest {                       // constructor compacto: normaliza
        codigo = codigo == null ? null : codigo.strip().toUpperCase(Locale.ROOT);  // "amox-500 " → "AMOX-
500"
    }
}
```

Un **record** es una clase corta e **inmutable** para transportar datos; Java genera el constructor, los métodos de lectura (`r.nombre()`), `equals`, `hashCode` y `toString`. Todos los DTO del proyecto son records. Las entidades son clases comunes porque JPA necesita crearlas vacías y modificarlas.

Enums: el único lugar de los permisos

```
public enum Rol {
    ADMIN(EnumSet.allOf(Permiso.class)),
    FARMACEUTICO(EnumSet.of(Permiso.LECTURA, Permiso.CATALOGO_EDITAR, Permiso.STOCK_INGRESAR,
                            Permiso.STOCK_BAJA_AJUSTE, Permiso.TRANSFERENCIA_CREAR)),
    CONSULTA(EnumSet.of(Permiso.LECTURA));
    ...
}
```

Otros enums: `TipoMovimiento` (INGRESO, EGRESO, BAJA_VENCIMIENTO, AJUSTE), `EstadoLote` (VIGENTE, POR_VENCER, VENCIDO, AGOTADO), `EstadoStock` (SIN_STOCK, STOCK_BAJO, OK) y `FormatoReporte` (CSV, XLSX, PDF).

Streams y comparadores

```
Map<Long, List<Lote>> lotes =
    loteRepository.bloquearDisponibles(cantidades.keySet(), hoy).stream()
        .filter(lote -> lote.tieneStock() && !lote.estaVencido(hoy))   // los útiles
        .sorted(ORDEN_FEFO)                                            // por vencimiento
        .collect(Collectors.groupingBy(lote -> lote.getMedicamento().getId()));

static final Comparator<Lote> ORDEN_FEFO =
    Comparator.comparing(Lote::getFechaVencimiento)   // primero por vencimiento
        .thenComparing(Lote::getFechaIngreso)         // empate: el que ingresó antes
        .thenComparing(Lote::getId);                  // empate: el id
```

Un **stream** es una cadena de montaje: `filter` (filtrar), `map` (transformar), `sorted` (ordenar), `toList` (juntar), `groupingBy` (agrupar). `Lote::getFechaVencimiento` es una referencia a método, igual a `lote -> lote.getFechaVencimiento()`.

Excepciones del dominio

Excepción	Cuándo	Se convierte en
`RecursoNoEncontradoException`	No existe lo buscado.	404 `NO_ENCONTRADO`
`ReglaNegocioException(codigo, mensaje)`	Viola una regla.	409 con ese código
`StockInsuficienteException(faltantes)`	No alcanza el stock.	409 `STOCK_INSUFICIENTE` + faltantes
`ErrorDeCampoException(campo, mensaje)`	Campo inválido según una regla del servicio.	400 `VALIDACION`
`ParametroInvalidoException`	Parámetro de la URL inválido.	400
`IllegalArgumentException` / `IllegalStateException`	Invariante de una entidad violada (no debería pasar).	500

Todas son *unchecked* (heredan de `RuntimeException`): no hace falta declararlas.

Anotaciones y Lombok

Las **anotaciones** (`@Algo`) son etiquetas que no hacen nada solas: las lee otro programa (Spring, JPA, Lombok, el validador). **Lombok** genera código repetitivo; en el proyecto solo `@Getter`, `@Setter` selectivo y `@RequiredArgsConstructor`. **Prohibido** `@Data`: puede imprimir relaciones enteras o el hash de la contraseña y romper con JPA.

`final` = se asigna una vez (las dependencias de un servicio son `private final`). `static` = pertenece a la clase (constantes y métodos como `AreaResponse.desde`).

> **Cómo leer una clase:** paquete → anotaciones de la clase → atributos `private final` (sus dependencias) → métodos públicos → comentarios. Probalo con `area/AreaService.java`.

6. Spring Boot

Un **framework** es un esqueleto ya armado: vos completás tu negocio y él se encarga del resto. **Spring Boot 4.1** (no 3.x, porque la 3.5 dejó de tener soporte gratuito en junio de 2026) levanta el servidor web, se conecta a la base, maneja la seguridad y convierte JSON.

Arranque

`SpringApplication.run(...)`:

1. Lee la configuración (`application.yml` + el perfil activo).
2. **Escanea** el paquete `ar.edu.utn.farmacia` buscando clases anotadas.
3. Crea un objeto de cada una (**beans**) y los conecta (**inyección de dependencias**).
4. Ejecuta Flyway.
5. Levanta el servidor en el puerto 8080.

Inyección de dependencias (lo más importante)

```
@Service
@RequiredArgsConstructor                        // constructor con los atributos final
public class AreaService {
    private final AreaRepository areaRepository; // Spring lo inyecta
}
```

La clase **pide** lo que necesita en el constructor y **Spring se lo pasa**. Si la clase creara su repositorio con `new`, tendría que saber cómo se construye y en las pruebas no se podría reemplazar por uno falso. Regla del proyecto: **por constructor**, nunca `@Autowired` en un atributo.

> **Para la defensa:** "Las clases no crean sus dependencias: las reciben en el constructor y Spring se las inyecta. Así cada parte se puede probar sola, reemplazando las demás por simulaciones."

Anotación	Capa	Ejemplo
`@RestController`	Recibe HTTP y devuelve JSON.	`AreaController`
`@Service`	Lógica de negocio.	`AreaService`
Repositorio (interfaz)	Acceso a datos.	`AreaRepository`
`@Component`	Cualquier otro bean.	`UsuarioJwtConverter`, `CorreoSimulado`
`@Configuration` + `@Bean`	Fabrica beans a mano.	`RelojConfig` crea el `Clock`.

Controladores

```
@RestController
@RequestMapping("/api/areas")
@RequiredArgsConstructor
public class AreaController {
    private final AreaService areaService;

    @PostMapping                                        // POST /api/areas
    @PreAuthorize("hasAuthority('CATALOGO_EDITAR')")   // sin permiso → 403
    public ResponseEntity<AreaResponse> crear(@Valid @RequestBody AreaRequest pedido) {
        AreaResponse creada = areaService.crear(pedido);
        return ResponseEntity.created(URI.create("/api/areas/" + creada.id())).body(creada);  // 201
    }
}
```

`@PathVariable` toma un valor de la ruta, `@RequestParam` uno de la URL, `@RequestBody` convierte el JSON en un record, `@Valid` lo valida (si falla, 400) y `ResponseEntity` permite elegir código y cabeceras. **El controlador no tiene lógica**: traduce HTTP ↔ Java y llama al servicio.

Servicios y transacciones

`@Transactional` abre una transacción al entrar y la confirma al salir; si hay una excepción, **revierte todo**. Las consultas usan `@Transactional(readOnly = true)`. Dentro de la transacción, si modificás un objeto que vino de la base no hace falta `save`: JPA detecta el cambio y hace el `UPDATE` solo. Por eso `cambiarEstado` es una línea: `buscar(id).setActivo(activo);`.

JPA, Hibernate y Spring Data

JPA es el estándar para mapear clases a tablas, **Hibernate** lo implementa y **Spring Data** genera los repositorios. Todo eso es un **ORM**.

```
@Entity @Table(name = "lote")
public class Lote {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)   // muchos lotes → un medicamento
    @JoinColumn(name = "medicamento_id")
    private Medicamento medicamento;

    protected Lote() {}   // JPA lo necesita para crear el objeto al leer de la base
}

public interface AreaRepository extends JpaRepository<Area, Long> {
    boolean existsByNombreIgnoreCase(String nombre);   // Spring arma la consulta por el nombre
}
```

- Extender `JpaRepository` ya da `findById`, `findAll`, `save`, `count`…
- `@Query` permite escribir **JPQL** (como SQL pero con clases y atributos Java), y `@Lock(PESSIMISTIC_WRITE)` lo convierte en `SELECT … FOR UPDATE`.
- El listado de medicamentos con stock usa **SQL directo** con `JdbcClient`, porque agrupa y ordena por valores calculados.
- **Specifications** arman el `WHERE` según los filtros que mandó el usuario.
- `open-in-view: false` obliga a armar los DTO dentro del servicio y evita consultas escondidas.

Validación y errores

Anotación	Regla
`@NotNull` / `@NotBlank`	No nulo / texto no vacío.
`@Size(min, max)`	Largo del texto o de la lista.
`@Positive` / `@PositiveOrZero`	> 0 / ≥ 0.
`@Email`	Formato de correo.
`@Cuit`	Propia: CUIT con dígito verificador (`CuitValidator`).

`ManejadorErrores` (`@RestControllerAdvice`) atrapa las excepciones de todos los controladores y las convierte en ProblemDetail. Un error inesperado devuelve solo "Ocurrió un error inesperado" con un `errorId`; el detalle va al log.

Configuración y perfiles

```
spring:
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
farmacia:
  zona-horaria: America/Argentina/Buenos_Aires
  jwt:
    secret: ${JWT_SECRET:}     # de la variable de entorno
    duracion: 8h
  alertas:
    dias-por-vencer: 30
```

`JwtProperties` es un record validado: si `secret` falta o tiene menos de 32 bytes, lanza una excepción y **la aplicación no arranca**. Los perfiles (`dev`, `demo`, `prod`, `test`) suman sus archivos `application-<perfil>.yml`, y `@Profile("demo")` activa clases solo en un perfil (como `DemoDataLoader`).

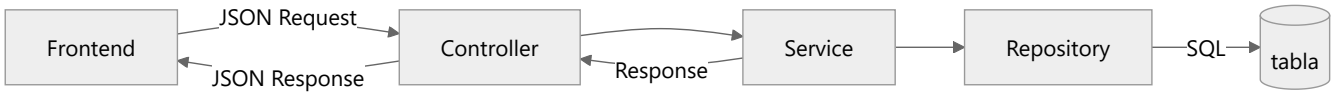
Otras piezas: Jackson (JSON), springdoc-openapi (Swagger), Actuator (solo `/actuator/health`), SLF4J (logs, nunca con contraseñas ni tokens) y Maven.

7. Recorrido del backend

Todo está en `backend/src/main/java/ar/edu/utn/farmacia/`, organizado **por funcionalidad**:

```
auth/           login, token, roles, permisos, recuperación, admin inicial, seguridad
usuario/        ABM de usuarios
proveedor/      ABM de proveedores + validación de CUIT
area/           ABM de áreas  ← el más simple: empezá por acá
medicamento/    catálogo + stock calculado con SQL
lote/           lotes, ingreso, baja, ajuste, movimientos
transferencia/  transferencias FEFO, comprobante PDF, reporte
alerta/         alertas y resumen del panel
common/         errores, reloj, reportes (CSV/Excel/PDF), utilidades
demo/           datos de ejemplo (solo perfil demo)
```

Cada carpeta repite el mismo patrón: `Area` (entidad), `AreaRepository`, `AreaService`, `AreaController`, `AreaRequest` (entrada con validaciones) y `AreaResponse` (salida con `desde(...)`).



Viaje 1: crear un área

1. **Filtros de seguridad**: validan el token y cargan los permisos. Si falla → 401.
2. **Controlador**: `@PreAuthorize` (sin permiso → 403), Jackson arma el `AreaRequest`, el constructor compacto limpia espacios y `@Valid` valida (vacío → 400).
3. **Servicio**: regla de nombre único (`NOMBRE_DUPLICADO` → 409).
4. **Repositorio y base**: `save` hace el `INSERT`. Además hay un índice único sobre `lower(nombre)`: si dos personas crean la misma área a la vez, la base frena la segunda.
5. **Respuesta**: **201** con `Location: /api/areas/9` y el JSON.

Viaje 2: ingreso de mercadería

`POST /api/medicamentos/5/lotes` con proveedor, número de lote, vencimiento y cantidad.

- Todas las validaciones van **antes** de modificar nada: medicamento y proveedor existen y están activos, el vencimiento no es pasado y el lote no está repetido.
- La fecha se valida **en el servicio con el reloj** y no con `@FutureOrPresent`, que usa el reloj del sistema (UTC).
- El usuario sale de `UsuarioActual` (del **token**), **nunca** del JSON.
- Se crean **dos filas** en la misma transacción: el lote y su movimiento `INGRESO`.

Viaje 3: una transferencia (el más importante)

```
@Transactional
public TransferenciaResponse transferir(TransferenciaRequest pedido) {
    Map<Long, Integer> cantidades = sumarItems(pedido.items());            // 1. sumar repetidos
    Area area = buscarAreaActiva(pedido.areaId());                         // 2. validar (404 / 409)
    Map<Long, Medicamento> medicamentos = buscarMedicamentosActivos(cantidades);
    Map<Long, List<Lote>> lotes = ...bloquearDisponibles(...)              // 3. bloquear + FEFO
    List<Faltante> faltantes = faltantes(cantidades, medicamentos, lotes); // 4. todo o nada
    if (!faltantes.isEmpty()) throw new StockInsuficienteException(faltantes);
    // 5. descontar lote por lote: detalle + movimiento EGRESO por cada lote tocado
    // 6. guardar y responder el comprobante (201)
}
```

No hay un `save` de cada lote: vinieron de la base dentro de la transacción, así que JPA hace el `UPDATE` solo al terminar.

`common/`, `alerta/`, `auth/` y `demo/`

Pieza	Para qué
`ManejadorErrores`, `Problemas`	Excepciones → ProblemDetail con `codigo`.
`RelojConfig`	El `Clock` con la zona de Buenos Aires.

Pieza	Para qué
`Ordenamiento`	Lista blanca del `sort` (si no, 400 `ORDEN_INVALIDO`).
`Reporte` + `Exportador`	Un solo modelo de reporte, tres formatos (CSV, XLSX, PDF): siempre muestran lo mismo.
`Csv`	Protege contra inyección de fórmulas.
`AlertaService`	Calcula vencidos, por vencer, sin stock y stock bajo en cada pedido; no guarda nada.
`ResumenService`	El panel: totales, transferencias de hoy, últimas 5 y unidades por área.
`SeguridadConfig`	Rutas públicas, BCrypt, firma y validación del JWT.
`UsuarioJwtConverter`	En cada pedido: token → usuario de la base → permisos.
`AdminInicial`	Crea `admin` al arrancar si no hay ningún ADMIN.
`DemoDataLoader`	Datos de ejemplo usando los propios servicios; idempotente; solo perfil `demo`.
`PerfilesIncompatibles`	Impide arrancar con `demo` y `prod` juntos.

8. Seguridad

Autenticación y autorización

- **Autenticación**: ¿**quién** sos? (usuario y contraseña → token)
- **Autorización**: ¿**qué** podés hacer? (rol → permisos)

Como en un edificio: en la entrada te piden el DNI y te dan una tarjeta que abre solo algunos pisos.

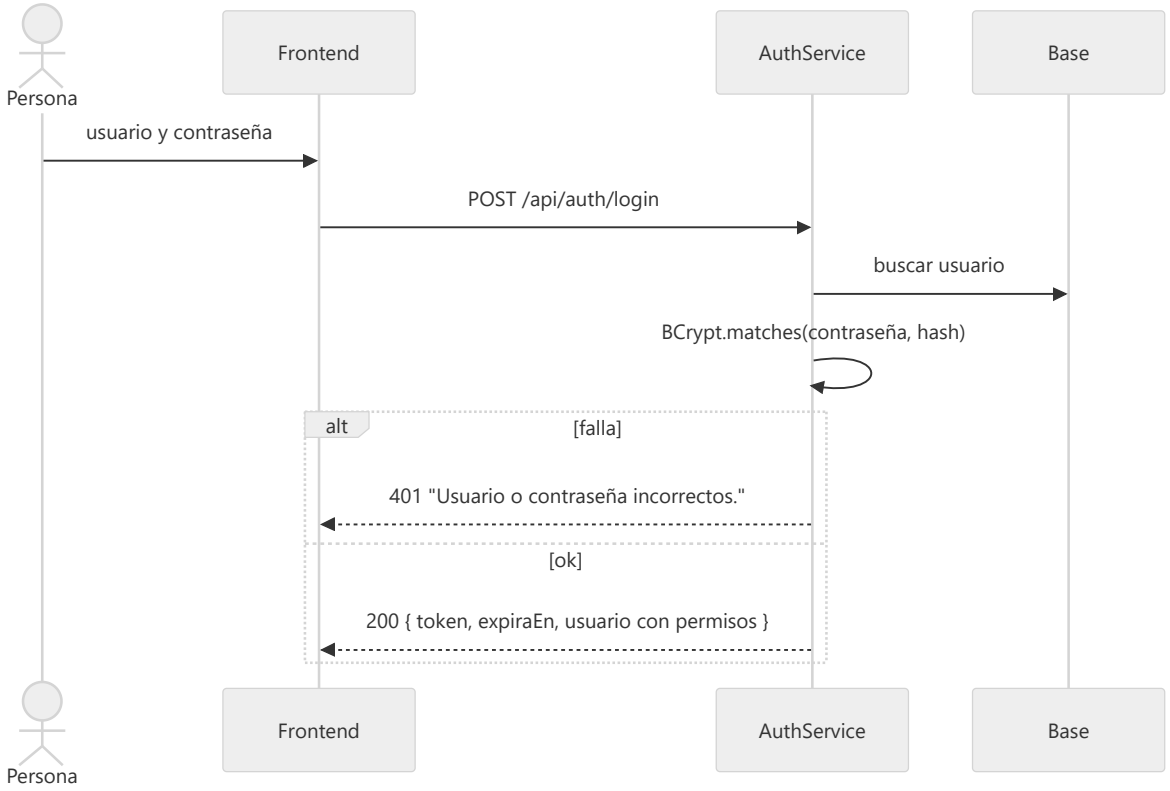
Contraseñas: hash con BCrypt

Si la base guardara las contraseñas tal cual y alguien la robara, tendría todas. Por eso se guarda un **hash**: siempre da lo mismo para la misma entrada, **no se puede volver atrás** y un cambio mínimo cambia todo el resultado. Para el login se calcula el hash de lo escrito y se compara.

> **Ojo:** hash **no** es encriptar. Lo encriptado se desencripta con la clave; un hash no se revierte.

BCrypt es lento a propósito (costo 12 = 4096 rondas) y agrega una **sal** aleatoria por usuario: dos personas con la misma contraseña tienen hashes distintos y un atacante no puede probar miles de millones por segundo. Contraseñas de 8 a 72 caracteres. La columna `password_hash` no está en ningún DTO.

El login



- **Mismo mensaje para todo error**, para no revelar qué usuarios existen. Aunque el usuario no exista, se calcula un hash igual, así el **tiempo** tampoco lo delata.
- El nombre de usuario no distingue mayúsculas.
- **No hay registro público**: solo un ADMIN crea usuarios.

El token JWT

Después del login el servidor no recuerda a nadie: le da al navegador un **token** firmado que se manda en cada pedido. Tiene tres partes en Base64: cabecera, datos y firma.

```
{ "alg": "HS256" }
{ "iss": "farmacia-hospitalaria", "sub": "2", "iat": 1762978000, "exp": 1763006800 }
```

- El contenido **no está cifrado**: cualquiera lo lee. Por eso **no lleva datos sensibles** (ni rol ni nombre), solo emisor, id del usuario y fechas (dura 8 horas).
- Lo que lo hace seguro es la **firma**: `HMAC-SHA256(cabecera.datos, JWT_SECRET)`. Si alguien cambia `"sub": "2"` por `"sub": "1"`, la firma ya no coincide → **401**.
- `JWT_SECRET` viene de una variable de entorno, con mínimo 32 bytes.

En cada pedido (`Authorization: Bearer …`) el servidor:

1. Verifica la **firma**.
2. Verifica `exp` con el reloj de la aplicación.
3. Verifica `iss`.
4. Busca el usuario por `sub` en la base: si no existe o está **inactivo** → 401.
5. Le asigna los **permisos de su rol actual**, leídos de la base.

> **Para la defensa:** "El token solo dice quién sos. Qué podés hacer se lee de la base en cada pedido. Por eso una baja o un cambio de rol tiene efecto inmediato. En el prototipo la identidad salía de una cabecera que mandaba el navegador y se podía falsificar."

Roles y permisos

El código pide **qué se necesita** (`hasAuthority('STOCK_INGRESAR')`) y no **quién** (`hasRole('ADMIN')`). Si mañana se crea un rol "Técnico", solo se toca el enum `Rol`.

	401 Unauthorized	403 Forbidden
Significa	No sé quién sos.	Sé quién sos, pero no podés.
Casos	Sin token, alterado, vencido, usuario desactivado.	CONSULTA intentando crear un medicamento.
El frontend	Borra la sesión y lleva al ingreso.	"No tenés permiso para esta acción."

Ocultar no es proteger: el frontend esconde los botones que tu rol no puede usar, pero eso es comodidad. Si alguien llama a la API con otro programa, igual recibe 403.

Recuperación de contraseña

1. Se pide un código con el usuario. La respuesta es **la misma exista o no** el usuario.
2. Se generan 6 números con `SecureRandom` (aleatorio criptográfico).
3. Se guarda el **hash** del código, con vencimiento a los 15 minutos; se anulan los anteriores.
4. El envío hoy es **simulado** (`CorreoSimulado` deja constancia en el log, sin el código); en `demo` el código se muestra en pantalla.
5. Se manda usuario + código + contraseña nueva; se verifica con BCrypt.
6. Máximo **5 intentos**. El contador se guarda **aunque la operación falle** (`noRollbackFor`) y se incrementa con bloqueo pesimista, para que no se pueda burlar.

Ataques comunes

Ataque	Qué es	Defensa
Inyección SQL	Meter SQL en un campo.	Consultas parametrizadas + lista blanca para el orden.
XSS	Guardar código que se ejecute en el navegador de otro.	React escapa el texto, ESLint prohíbe `innerHTML`, CSP en nginx y `seguridad.test.tsx`.
Fórmulas en CSV	Un dato que empieza con `=` se ejecuta en Excel.	`Csv` antepone `'` a valores que empiezan con `=`, `+`, `-`, `@`.
CSRF	Otra página hace que tu navegador mande un pedido con tus cookies.	No aplica: no hay cookies, el token va en `Authorization`.
CORS	Leer respuestas de otro origen.	Mismo origen detrás del proxy; nunca `*`.
Fuerza bruta	Probar contraseñas una tras otra.	BCrypt hace lento cada intento; límite de intentos queda como mejora.

Dónde guarda el token el frontend

Opción	Ventaja	Desventaja
`sessionStorage` (elegida)	Se borra al cerrar la pestaña (PC compartidas).	JavaScript puede leerlo → hay que prevenir XSS.
`localStorage`	Sobrevive a cerrar el navegador.	Queda guardado en PC compartidas.
Cookie HttpOnly	JavaScript no la lee.	El navegador la manda sola → hay que proteger CSRF.

Al recargar, el frontend no confía en lo guardado: pide `/api/auth/me` para tener permisos frescos.

Configuración segura

Secretos solo en variables de entorno (`.env` no se sube a git); la app no arranca sin `JWT_SECRET` válido ni con `demo` + `prod`; Swagger apagado en producción; Actuator solo `/health`; los 500 no muestran detalles internos; nunca se loguean contraseñas ni tokens; puertos solo en `127.0.0.1`; contenedores sin root.

Los 8 problemas del prototipo, resueltos

#	Problema	Solución	Prueba
1	Identidad en una cabecera falsificable	Identidad solo del JWT; permisos de la base	`SeguridadIT`
2	Clave del JWT en el código	Variable de entorno ≥ 32 bytes	`ConfiguracionTest`
3	Stock negativo	Validación + entidad + CHECK + bloqueo	`TransferenciaConcurrenciaIT`, `LoteTest`
4	Hash expuesto	Solo DTO, sin hash	`SeguridadIT`
5	Registro libre	Sin registro; solo ADMIN crea	`SeguridadIT`
6	Fechas como texto	Tipos de fecha reales + `Clock`	`AlertaServiceTest`
7	Borrado en cascada	Bajas lógicas, FK sin cascada	`ProveedorServiceTest`
8	Errores genéricos, sin tests	ProblemDetail con código; pruebas en `verify`	`ErroresIT`

9. Reglas de negocio: FEFO, stock y concurrencia

Es **el corazón del sistema**. Si entendés esta sección, entendés por qué el sistema existe.

Estados de un lote

Hoy es **12/11/2026** y "por vencer" son 30 días (hasta el 12/12/2026).

Lote	Vence	Cantidad	Estado	Por qué
A	11/11/2026	20	**VENCIDO**	Venció ayer (fecha < hoy).
B	12/11/2026	50	**POR_VENCER**	Vence **hoy**: todavía se puede usar.
C	12/12/2026	80	**POR_VENCER**	Vence justo en 30 días (incluido).
D	13/12/2026	100	**VIGENTE**	Vence en 31 días.

Lote	Vence	Cantidad	Estado	Por qué
E	01/06/2027	0	**AGOTADO**	Sin unidades (se mira primero).

```
public EstadoLote estado(LocalDate hoy, int diasPorVencer) {
    if (cantidadActual == 0)                                    return EstadoLote.AGOTADO;
    if (estaVencido(hoy))                                       return EstadoLote.VENCIDO;    // fecha <
hoy
    if (!fechaVencimiento.isAfter(hoy.plusDays(diasPorVencer))) return EstadoLote.POR_VENCER; // fecha <=
hoy + 30
    return EstadoLote.VIGENTE;
}
```

El estado **no se guarda**: cambia solo con el paso del tiempo. Los bordes tienen pruebas propias (`AlertaServiceTest`) con reloj fijo.

Stock total y estado del medicamento

Stock total = lotes no vencidos: B + C + D + E = 50 + 80 + 100 + 0 = **230**. El lote A (20, vencido) está en el depósito pero no se puede entregar. Con stock mínimo 300: 0 es SIN_STOCK, de 1 a 300 es STOCK_BAJO (230 cae acá) y más de 300 es OK. El **próximo vencimiento** es el menor entre los lotes con stock no vencidos (lote B).

FEFO: primero vence, primero sale

Lote	Ingresó	Vence
X	01/09	01/06/2027
Y	01/10	15/12/2026

FIFO entregaría X (entró antes) y Y se vencería en el estante. **FEFO** entrega Y, que vence antes. Desempates: fecha de ingreso y después id, para que el resultado sea siempre el mismo (**determinístico**).

Ejemplo completo

Paracetamol, hoy 12/11/2026. Guardia pide **150**.

Lote	Vence	Cantidad
L2290	10/11/2026	30 ← vencido, se ignora
L2301	01/12/2026	100
L2340	15/03/2027	410
L2350	15/03/2027	500 (ingresó después que L2340)

1. Se descarta el vencido.
2. Orden FEFO: L2301 → L2340 → L2350.
3. Disponible: 1010 ≥ 150, alcanza.
4. L2301: toma 100 (queda 0, faltan 50). L2340: toma 50 (queda 360). Fin.

Lote	Tipo	Cantidad	Saldo resultante
L2301	EGRESO	−100	0

Lote	Tipo	Cantidad	Saldo resultante
L2340	EGRESO	−50	360

El comprobante tiene dos renglones de detalle, uno por cada lote tocado.

Todo o nada

Pedido: Paracetamol 150 (hay 1010), Amoxicilina 50 (hay 12) y Ceftriaxona 20 (hay 5). El sistema **primero calcula todos los faltantes** y recién después toca algo. Como faltan dos, lanza `StockInsuficienteException` con **los dos**, la transacción se revierte y **no se descuenta nada, ni siquiera el Paracetamol**:

```
{ "status": 409, "codigo": "STOCK_INSUFICIENTE",
  "faltantes": [
    { "medicamento": "Amoxicilina comprimidos 500 mg", "disponible": 12, "solicitado": 50 },
    { "medicamento": "Ceftriaxona frasco ampolla 1 g", "disponible": 5, "solicitado": 20 } ] }
```

El frontend marca cada línea ("Disponible: 12 (pediste 50)") y **conserva todo lo cargado**. ¿Por qué todo o nada? Porque una entrega a medias confunde: el área cree que recibió todo y el comprobante no coincide con lo físico.

Ítems repetidos: Paracetamol 100 + Paracetamol 50 se transforma en Paracetamol 150, tanto en el backend como en la pantalla.

Concurrencia: dos personas al mismo tiempo

Lote L2301 con **100**. Ana y Bruno transfieren **60** cada uno en el mismo segundo.

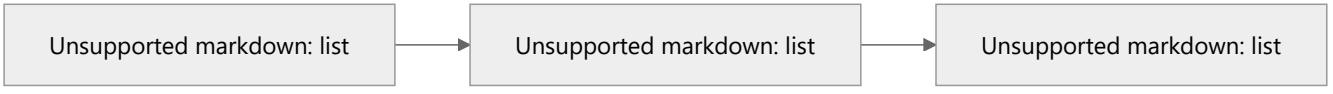
```
Sin protección:
Ana:   lee 100 ── 100 ≥ 60 ✓ ── guarda 40
Bruno: lee 100 ── 100 ≥ 60 ✓ ── guarda 40      → se entregaron 120 ❌

Con bloqueo (SELECT ... FOR UPDATE):
Ana:   BLOQUEA y lee 100 ── guarda 40 ── confirma (libera)
Bruno: ESPERA ──────────────────────────── lee 40 ── 40 < 60 ── 409   ✅
```

El **deadlock** se evita porque la consulta **ordena por id** y bloquea **todos los lotes en una sola consulta**: los dos intentan primero el lote de menor id, uno lo consigue y el otro espera sin tener nada bloqueado. Lo demuestra `TransferenciaConcurrenciaIT` contra PostgreSQL real (un 201, un 409, stock final 40) y `pedidosEnOrdenInversoNoSeTraban`.

> **Para la defensa:** "Bloqueo los lotes con `SELECT … FOR UPDATE` en una sola consulta ordenada por id. El bloqueo evita que dos transferencias usen el mismo stock y el orden fijo evita el deadlock. Lo demuestra una prueba que lanza dos transferencias simultáneas contra PostgreSQL."

Stock nunca negativo: tres barreras



Si una capa tuviera un error, la siguiente lo detiene: **defensa en profundidad**. Más el bloqueo para la concurrencia.

Trazabilidad

Cada cambio de stock genera **exactamente un movimiento**. Por eso siempre se cumple:

> **suma de los movimientos de un lote = cantidad actual del lote**

Fecha	Tipo	Cantidad	Saldo
01/10	INGRESO	+410	410
12/11	EGRESO (TR-000042)	−50	360
13/11	AJUSTE ("rotura de 2 cajas")	−20	340
Suma		**340**	= cantidad actual ✅

`FlujoCompletoIT` verifica esta igualdad para cada lote.

Baja por vencimiento y ajuste

Situación	Resultado
Baja de un lote vencido con 20	Queda en 0; movimiento `BAJA_VENCIMIENTO` −20 con motivo.
Baja de un lote que vence mañana	409 `LOTE_NO_VENCIDO`.
Baja de un lote vencido ya en 0	409 `LOTE_SIN_STOCK`.
Ajuste de 360 a 340	`AJUSTE` −20.
Ajuste de 0 a 5	`AJUSTE` +5 (se encontraron cajas).
Ajuste a más que la cantidad inicial	400.
Ajuste al mismo valor	400: tiene que ser distinto.

El ajuste también usa bloqueo pesimista sobre el lote.

Alertas

Alerta	Condición	Orden
VENCIDO	Lote con stock y vence < hoy (aunque el medicamento esté inactivo).	Por fecha.
POR_VENCER	Lote con stock y hoy ≤ vence ≤ hoy + 30.	Por fecha.
SIN_STOCK	Medicamento **activo** con stock 0.	Por nombre.
STOCK_BAJO	Medicamento **activo** con 0 < stock ≤ mínimo.	Más crítico primero (stock ÷ mínimo).

Ejemplo: Amoxicilina 50/200 = 0,25 y Ceftriaxona 70/80 = 0,875 → primero Amoxicilina.

Otras reglas

Regla	Por qué
Medicamento con stock no vencido no se desactiva (`MEDICAMENTO_CON_STOCK`).	Quedaría mercadería invisible.

Regla	Por qué
Proveedor con stock **sí** se desactiva.	Dejar de comprarle no hace desaparecer su mercadería.
Área inactiva no recibe transferencias (`AREA_INACTIVA`).	Si el servicio cerró, no se le entrega.
Nadie se desactiva a sí mismo (`PROPIO_USUARIO`).	Evita quedarse afuera por error.
No se desactiva ni se le quita el rol al último ADMIN (`ULTIMO_ADMIN`).	Siempre alguien gestiona usuarios.
Las transferencias no se editan ni se anulan.	El historial tiene que ser confiable; se corrige con ajustes.
Código de medicamento en mayúsculas.	Que `amox-500` y `AMOX-500` no convivan.

El dígito verificador del CUIT

Para `20-12345678-?` se multiplica cada dígito por los pesos 5, 4, 3, 2, 7, 6, 5, 4, 3, 2:

Dígito	2	0	1	2	3	4	5	6	7	8
Peso	5	4	3	2	7	6	5	4	3	2
Producto	10	0	3	4	21	24	25	24	21	16

Suma = 148. 148 mod 11 = 5. DV = 11 − 5 = **6** → `20-12345678-6`. Si da 11, el DV es 0; si da 10, el CUIT es inválido. Backend y frontend usan los mismos ejemplos en sus pruebas.

La hora de Argentina

Los contenedores corren en **UTC** (3 horas adelante). A las **21:00 del 12/11** en Buenos Aires, en UTC ya son las **00:00 del 13/11**: un lote que vence el 12/11 aparecería vencido tres horas antes. Por eso "hoy" sale siempre de un `Clock` con zona `America/Argentina/Buenos_Aires`. En el frontend, `new Date("2026-11-30")` se interpreta como medianoche UTC y mostraría **29/11**: las fechas sin hora se manejan como texto (`utils/fechas.ts`).

10. JavaScript y TypeScript

JavaScript es el lenguaje de los navegadores. **TypeScript** es JavaScript **con tipos**: se compila a JavaScript y el compilador avisa los errores antes. En el proyecto está en modo **estricto**, y los tipos de los datos del backend se **generan** desde su contrato OpenAPI (`src/api/esquema.ts`): si el backend cambia un campo, el frontend deja de compilar.

```
type EstadoStock = 'SIN_STOCK' | 'STOCK_BAJO' | 'OK';   // unión: uno de esos tres textos
type Medicamento = {
  id: number;
  nombre: string;
  stockTotal: number;
  estadoStock: EstadoStock;
  proximoVencimiento: string | null;   // texto o null
};
```

Concepto	Ejemplo	Qué hace
`const` / `let`	`const nombre = 'Amoxicilina'`	No se reasigna / se reasigna. `var` no se usa.
Función flecha	`(m) => m.nombre`	Función corta.

Concepto	Ejemplo	Qué hace
Desestructurar	`const { nombre, stock } = m`	Sacar campos a variables.
Spread	`{ ...m, stock: 0 }`, `[...lista, 4]`	Copiar sin modificar el original.
`map` / `filter` / `find` / `some` / `reduce`	`lista.filter((n) => n > 1)`	Transformar, filtrar, buscar, preguntar, acumular.
`?.`	`usuario?.permisos`	Si es null, da undefined en vez de error.
`??`	`valor ?? 'por defecto'`	Valor por defecto ante null.
Template string	`` `Stock de ${nombre}` ``	Texto con valores adentro.

> **Inmutabilidad:** en React no se modifican los objetos ni las listas del estado; se crean copias con `...`. Así React se da cuenta de que algo cambió.

Asincronía

Pedir datos al servidor tarda. JavaScript no se queda esperando (la pantalla se congelaría): una **Promise** avisa cuando llega la respuesta. `async` marca una función que devuelve una promesa y `await` espera su resultado. En el proyecto casi nunca se escribe `await` en las pantallas: **TanStack Query** se encarga, y el único `fetch` está en `src/api/cliente.ts`.

APIs del navegador que usa el proyecto

API	Para qué
`fetch`	Pedidos HTTP (solo en `cliente.ts`).
`AbortController`	Cortar un pedido a los 15 s o al cerrar la pantalla.
`sessionStorage`	El token y el borrador de la transferencia.
`Intl.NumberFormat` / `Intl.DateTimeFormat`	Números (`1.234`) y fechas (`30/11/2026`) en formato argentino.
`URLSearchParams`	Filtros en la URL.
`Blob` + `URL.createObjectURL`	Descargar Excel y PDF.
`<dialog>` + `showModal()`	Modales.
`setTimeout`	Avisar 5 minutos antes de que venza la sesión.

Herramientas: Node.js (ejecuta las herramientas), npm (instala y corre comandos), Vite (servidor de desarrollo y build), tsc (tipos), ESLint (prohíbe `innerHTML` y `console.log`), Prettier (formato), knip (código sin usar) y Vitest (pruebas).

11. React

React construye interfaces con **componentes**, piezas reutilizables que se combinan como bloques de Lego. Sin React habría que buscar elementos, borrarlos y crearlos a mano cada vez que cambia algo. Con React se **describe cómo tiene que verse la pantalla según los datos**.

> **Idea central:** pantalla = función(datos). Cambian los datos → React redibuja lo necesario.

Componentes, JSX y props

```
type PropsInsignia = { texto: string; tono: 'ok' | 'aviso' | 'peligro' | 'neutro' };

export function Insignia({ texto, tono }: PropsInsignia) {
  return <span className={`insignia insignia--${tono}`}>{texto}</span>;
}

<Insignia texto="Vencido" tono="peligro" />
```

- Un **componente** es una función que devuelve lo que se muestra; el nombre empieza con mayúscula.
- **JSX**: HTML dentro de TypeScript. `className` en lugar de `class`; `{}` para valores; `{cargando && <Cargando />}` para mostrar algo solo si se cumple; `lista.map(...)` con `key` para repetir.
- Todo lo que va entre `{}` se muestra **como texto**: React lo escapa. Por eso XSS es muy difícil, siempre que no se use `dangerouslySetInnerHTML` (prohibido por ESLint).
- Las **props** son los parámetros de un componente; van de padre a hijo. `children` es lo que va entre las etiquetas.

Estado y hooks

```
const [valor, setValor] = useState(0);    // valor actual + función para cambiarlo
setLineas([...lineas, nueva]);            // nunca lineas.push(nueva)
```

Hook	Para qué	En el proyecto
`useState`	Guardar estado.	Qué modal está abierto.
`useEffect`	Hacer algo después de dibujar.	Abrir el `<dialog>`; programar el aviso de sesión.
`useRef`	Guardar algo sin redibujar o referenciar un elemento.	Evitar el doble envío de una transferencia.
`useMemo` / `useCallback`	Recordar un cálculo o una función.	Valor del contexto de sesión.
`useContext`	Leer datos compartidos.	`useSesion()`, `useAvisos()`.
`useId`	Ids únicos para accesibilidad.	`aria-labelledby` del modal.

Hooks propios: `useFormulario` (valores, errores, validación al salir y al enviar), `useFiltrosUrl` (filtros en la URL), `useDebounce` (espera 300 ms antes de buscar), `useDescarga` (descarga con "Descargando…") y los de `api/` (`useMedicamentos`, `useAreas`…). Los hooks solo se llaman en el nivel superior del componente, nunca dentro de un `if` o un bucle.

Contexto

Datos que necesitan muchas pantallas (el usuario, los avisos) se ponen en un **contexto** en lugar de pasarlos por props de componente en componente:

```
const { puede } = useSesion();
{puede('CATALOGO_EDITAR') && <Link to="/medicamentos/nuevo">Nuevo medicamento</Link>}
```

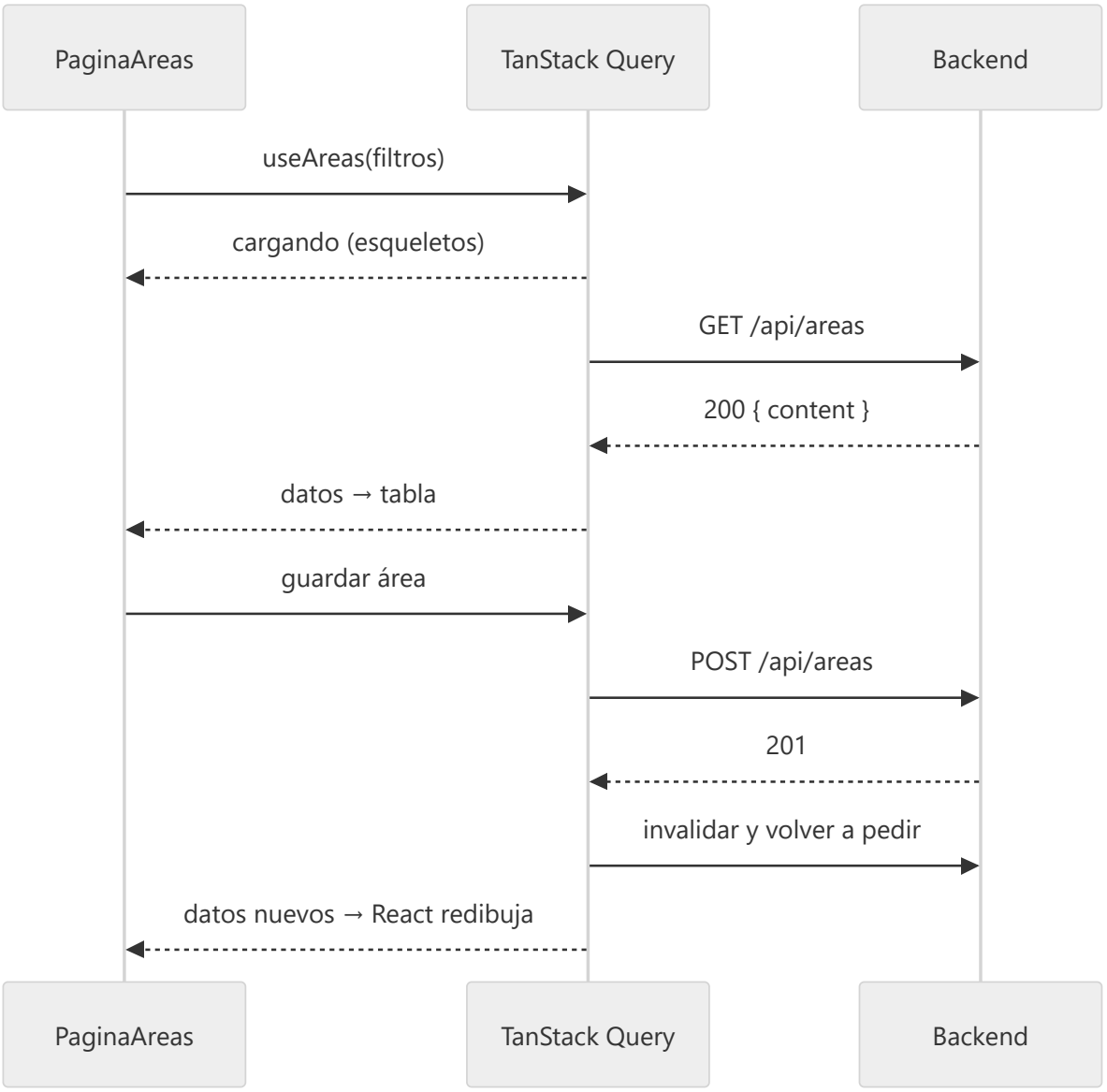
TanStack Query

```
export function useAreas(filtros: FiltrosAreas) {
  return useQuery({
    queryKey: ['areas', filtros],                                          // id en la caché
    queryFn: ({ signal }) => api.get('/api/areas', filtros, signal),      // cómo pedirlo
    placeholderData: keepPreviousData,                                     // mostrar la página anterior
mientras carga
  });
}
```

- **Caché**: si otra pantalla pidió lo mismo hace menos de 30 segundos, no se vuelve a pedir.
- **Reintento**: una sola vez y solo ante red caída o 5xx (un 400 o 404 no se arregla reintentando).
- `useMutation` **+ invalidar**: después de crear un área se marca `['areas']` como viejo y se vuelve a pedir. Después de un cambio de stock se invalidan medicamentos, lotes, movimientos, alertas, resumen, transferencias y proveedores.

React Router y formularios

- `createBrowserRouter` define las rutas; `RutaProtegida` exige sesión (y permiso si se indica).
- `<Link>` navega sin recargar; `useParams` lee `:id`; `useNavigate` navega desde el código; `<Outlet />` es el lugar de la ruta hija; `<Navigate>` redirige.
- Formularios: `e.preventDefault()` evita recargar, `noValidate` desactiva la validación del navegador (se usa la propia, en español) y `CampoFormulario` pone etiqueta, "(obligatorio)" y el error con `aria-invalid` y `aria-describedby`. Si el servidor responde 400 con `errors`, el mensaje aparece debajo del campo.



12. Recorrido del frontend

Todo está en `frontend/src/`:

```
main.tsx      arranca React e importa estilos
App.tsx       rutas + proveedores (sesión, avisos, consultas, límite de errores)
api/          hablar con el backend
auth/         sesión, ingreso, recuperación, rutas protegidas
components/   piezas reutilizables
hooks/        lógica reutilizable
pages/        una carpeta por pantalla
utils/        funciones puras (fechas, números, CUIT, validaciones)
styles/       CSS
```

`App.tsx` arma las capas: `LimiteErrores` (si algo explota, un mensaje en vez de pantalla en blanco) → `AvisosProvider` → `QueryClientProvider` → `RouterProvider` → `SesionProvider` → `RutaProtegida` → `Layout` (menú + contenido) → la página.

`api/`: la única puerta al backend

`cliente.ts` arma la URL, agrega `Authorization: Bearer <token>`, pone un máximo de **15 segundos** y traduce los errores a un `ErrorApi` en español:

Respuesta	Qué hace
401 en el login	"Usuario o contraseña incorrectos."
401 en otro lado	Borra la sesión → ingreso con "Tu sesión expiró".
403	"No tenés permiso para esta acción." y refresca los permisos.
400 / 404 / 409	El `detail` del backend, con `codigo`, `errors` y `faltantes`.
5xx	"Ocurrió un error en el servidor. Código: {errorId}."
Sin conexión	"No se pudo conectar con el servidor."
15 s sin respuesta	"El servidor tardó demasiado en responder."

> **Por qué un solo lugar:** el token, los errores y el tiempo máximo se manejan igual en toda la app. En el prototipo cada pantalla hacía su propio `fetch` y cada una fallaba distinto.

El cliente no importa React: recibe dos funciones del `SesionProvider`, así se prueba solo. `errores.ts` lee el ProblemDetail **sin confiar en su forma**. `esquema.ts` se genera con `npm run api:tipos` y **no se edita a mano**; `tipos.ts` le da nombres cortos. Hay un archivo por recurso (`medicamentos.ts`, `transferencias.ts`…) que exporta hooks.

`auth/`, `components/` y `utils/`

- **Sesión**: `almacenSesion.ts` guarda el token en `sessionStorage`; `SesionProvider` pide `/api/auth/me` al iniciar, expone `usuario`, `puede`, `iniciarSesion` y `cerrarSesion`, y avisa 5 minutos antes del vencimiento; `RutaProtegida` lleva a `/ingresar` recordando a dónde ibas.
- **Componentes**: `Layout`, `Menu` (cada enlace solo si `puede(permiso)`), `Tabla` (columnas declarativas, `aria-sort`, esqueletos), `Paginador`, `Modal` (`<dialog>` nativo), `Confirmacion`, `Avisos` (toasts), `Estados` (cargando, vacío, error con "Reintentar"), `Insignia`, `CampoFormulario`, `CampoBusqueda`, `BuscadorMedicamento` (combobox accesible) y `LimiteErrores`.
- **Utils** (funciones puras, fáciles de probar): `fechas.ts` (formato sin `new Date`, "hoy" en Buenos Aires), `numeros.ts`, `cuit.ts`, `estadoStock.ts`, `validaciones.ts` y `descargas.ts`.

La pantalla más compleja: Nueva transferencia

- **Borrador** en `sessionStorage`: se guarda en cada cambio y se valida al leerlo.
- Un medicamento repetido se **suma** a su línea.
- Cada cantidad tiene que ser entera, > 0 y ≤ disponible.
- **Revisar y confirmar** abre un modal con el resumen.
- Un `useRef` bloquea el **doble clic** (el segundo puede llegar antes de que el botón se deshabilite).
- Ante 409 `STOCK_INSUFICIENTE`: cierra el modal, marca cada línea con "Disponible: X (pediste Y)" y **conserva todo**.
- Si sale bien: borra el borrador, invalida las consultas de stock y va al comprobante.

Estilos y accesibilidad

CSS propio, sin Bootstrap ni Tailwind: `tokens.css` (variables de colores, espacios, tiempos), `base.css`, `componentes.css`, `animaciones.css` (con "reducir movimiento" quedan como fundidos) e `impresion.css` (el comprobante en blanco y negro). Nombres BEM: `.boton--primario`, `.menu__enlace`.

Pauta	Cómo se cumple
Contraste	Colores verificados contra WCAG AA.
Teclado	Todo usable con Tab, Enter y Escape; foco siempre visible.
Etiquetas	Todos los campos con `<label>`; errores con `aria-invalid` y `aria-describedby`.
No solo color	Los estados siempre llevan una palabra.
Modales y avisos	`<dialog>` con `aria-labelledby`; `role="status"` / `role="alert"`.
Movimiento	Respeta `prefers-reduced-motion`.
Verificación	`eslint-plugin-jsx-a11y` + axe en la prueba punta a punta.

Mock: `frontend/mock/servidor.mjs` es un backend simulado con datos en memoria que respeta el mismo contrato, para trabajar sin Java ni base. No entra en el build.

13. Docker y entornos

"En mi máquina anda": para correr el sistema hace falta Java 21, PostgreSQL 16, variables de entorno… **Docker** empaqueta cada programa con todo lo que necesita en un **contenedor** que corre igual en Windows, Linux o macOS.

Concepto	Analogía	Qué es
Imagen	La receta	Paquete de solo lectura (`postgres:16-alpine`).
Contenedor	La torta	Una imagen ejecutándose.
Dockerfile	La receta escrita	Pasos para construir una imagen.
Volumen	Disco externo	Datos que sobreviven al contenedor (la base).
Puerto publicado	Ventanilla	`127.0.0.1:8080:8080`.
Docker Compose	Plano de la cocina	Levanta varios contenedores juntos.

> **Ojo:** un contenedor **no** es una máquina virtual: comparte el núcleo del sistema operativo, por eso arranca en segundos.

Compose y Dockerfile

- El `docker-compose.yml` levanta `db` (PostgreSQL con volumen y *healthcheck*), `api` (espera a que la base esté sana) y `web` (nginx con el frontend). Dentro de Compose los servicios se encuentran **por nombre**: el backend se conecta a `db:5432`, no a `localhost`.
- La imagen de la base tiene versión fija (nunca `latest`) y `DB_PASSWORD` es obligatoria.
- **Dockerfile multi-etapa**: la primera etapa compila con el JDK; la segunda ejecuta solo con el JRE. La imagen final no lleva compilador ni código fuente: más chica y más segura.

- **Dependencias primero**: si solo cambia el código, Docker reusa la capa con las dependencias.
- `-DskipTests`: única excepción a no saltear pruebas, porque dentro del build no hay Docker para Testcontainers. Las pruebas corren con `./mvnw verify` y en la CI.
- **Usuario sin root** en los contenedores. El frontend se sirve con `nginx-unprivileged`.

Variables de entorno y perfiles

`.env.example` es la plantilla **sin secretos** (va a git); `.env` tiene los valores reales y **no** va a git.

Variable	Para qué
`DB_USER`, `DB_PASSWORD`, `DB_PUERTO`	Base de datos.
`JWT_SECRET`	Firmar los tokens (≥ 32 bytes).
`ADMIN_PASSWORD`	Contraseña del `admin` inicial.
`DEMO_PASSWORD`	Contraseña de los usuarios demo.
`SPRING_PROFILES_ACTIVE`	`demo`, `prod`, `dev`.

Perfil	Cuándo	Qué cambia
`dev`	Programando.	Conexión por defecto, SQL en el log, Swagger.
`demo`	Para mostrar.	Datos de ejemplo, código de recuperación en pantalla, Swagger.
`prod`	En el hospital.	Sin datos de ejemplo, sin Swagger.
`test`	Pruebas.	Reloj fijo, base en Testcontainers.

Comandos y levantar todo

Comando	Qué hace
`docker compose up --build`	Construye y levanta todo (`Ctrl + C` para parar).
`docker compose up -d --build`	Igual, en segundo plano.
`docker compose ps`	Qué está corriendo y si está sano.
`docker compose logs -f api`	Logs del backend en vivo.
`docker compose down`	Para y borra los contenedores (los datos quedan).
`docker compose down -v`	Igual y **borra los datos**.
`docker compose exec db psql -U farmacia farmacia`	Consola SQL dentro de la base.

1. Abrir Docker Desktop.
2. Copiar `.env.example` a `.env` y completar `DB_PASSWORD`, `JWT_SECRET` y `ADMIN_PASSWORD`.
3. `docker compose up --build` y esperar `Started FarmaciaApplication`.
4. Probar `http://localhost:8080/actuator/health` → `{"status":"UP"}`.
5. Abrir la aplicación e ingresar con `farmaceutico` / `demo1234` o `admin` / tu `ADMIN_PASSWORD`.

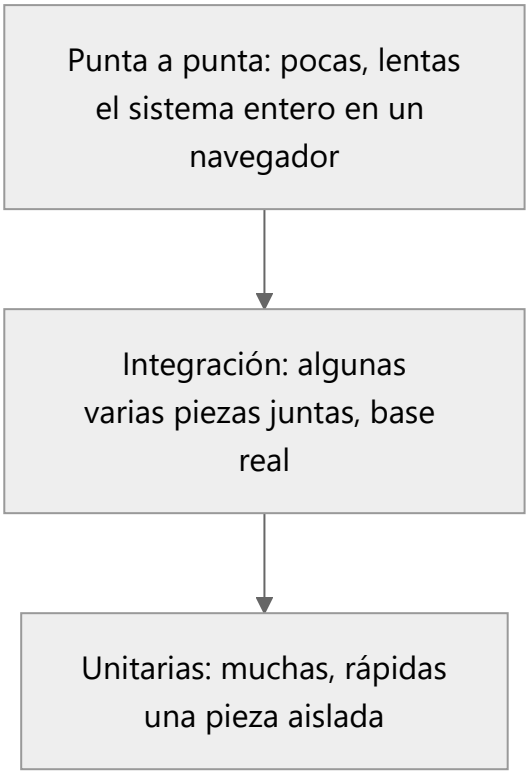
Integración continua: en cada push, GitHub Actions levanta una máquina limpia y corre `./mvnw verify` (formato + todas las pruebas del backend) y los chequeos del frontend. Si algo falla, se ve en rojo.

14. Pruebas

Una **prueba automática** es código que usa tu código y verifica el resultado. Detectan errores antes, permiten cambiar con confianza, **documentan** el comportamiento y **demuestran** cosas difíciles de ver a mano (dos transferencias al mismo milisegundo).

```
@Test
void unLoteQueVenceHoyNoEstaVencido() {
    LocalDate hoy = LocalDate.of(2026, 11, 12);   // Preparar
    Lote lote = loteQueVence(hoy);
    boolean vencido = lote.estaVencido(hoy);      // Actuar
    assertThat(vencido).isFalse();                // Verificar
}
```

El nombre dice qué comportamiento se espera: si falla, ya explica qué se rompió.



Tipo	Qué prueba	Ejemplo
Unitaria	Una clase o función sola, con simulaciones. Milisegundos.	`LoteTest`, `TransferenciaServiceTest`, `fechas.test.ts`
Integración	Controlador + servicio + base real. Segundos.	`FlujoCompletoIT`, `SeguridadIT`
Punta a punta	El sistema completo en un navegador.	`e2e/capturas.spec.ts`

Backend

Herramienta	Para qué
JUnit 5	Marca y ejecuta las pruebas.
AssertJ	Verificaciones legibles: `assertThat(stock).isEqualTo(40)`.

Herramienta	Para qué
Mockito	**Mocks**: objetos falsos que reemplazan dependencias.
MockMvc	Simula pedidos HTTP sin levantar un servidor.
Testcontainers	Levanta un PostgreSQL real en Docker.
JaCoCo	Mide la cobertura.

- `*Test` = unitarias (Surefire); `*IT` = integración (Failsafe). `./mvnw verify` corre las dos.
- **Mocks**: `TransferenciaServiceTest` prueba el FEFO sin base, con un repositorio falso que devuelve los lotes que la prueba quiere. Por eso importa la inyección por constructor.
- **Reloj fijo** (`RelojFijoConfig`): "hoy" es siempre la misma fecha.
- **¿Por qué PostgreSQL real y no H2?** Para probar lo que hace PostgreSQL de verdad: `CHECK`, `FOR UPDATE` y SQL específico. Todas las IT heredan de `IntegracionBase` y comparten un solo contenedor y un solo contexto.
- **La prueba de concurrencia**: crea un lote de 100, dos hilos piden 60 cada uno y un `CountDownLatch` los larga a la vez. Verifica exactamente un 201, un 409 y stock final 40. No es `@Transactional`, porque si lo fuera no habría concurrencia real.

Frontend

Vitest (ejecuta), Testing Library (dibuja un componente y lo usa como una persona), jsdom (navegador simulado), user-event (clics y escritura realistas), Playwright (Chrome real) y axe-core (accesibilidad). Se busca por **rol y texto visible** (`getByRole('button', { name: … })`): si la prueba encuentra el botón, también lo encuentra un usuario con lector de pantalla.

La prueba punta a punta abre Chrome, ingresa como admin, recorre **16 pantallas**, corre axe en cada una (falla ante problemas serios) y guarda una captura de cada pantalla.

TDD

Rojo (escribir una prueba que falla) → **verde** (el código mínimo para que pase) → **refactorizar** (mejorar con la prueba como red). Se usó en FEFO, alertas, CUIT, reglas del lote, el cliente HTTP y la nueva transferencia.

Si te preguntan…	La prueba es…
¿Funciona el FEFO?	`TransferenciaServiceTest` y `FlujoCompletoIT`.
¿Y con dos personas a la vez?	`TransferenciaConcurrenciaIT`.
¿No se ve el hash?	`SeguridadIT.ningunaRespuestaExponeLaContrasenaNiElHash`.
¿Y los bordes de vencimiento?	`AlertaServiceTest`.
¿Y si falta la clave del JWT?	`ConfiguracionTest`.
¿Los movimientos cierran con el stock?	`FlujoCompletoIT`.
¿El CUIT se valida igual en front y back?	`CuitValidatorTest` y `cuit.test.ts`.
¿Es accesible?	`capturas.spec.ts` con axe.
¿Un texto malicioso rompe la pantalla?	`seguridad.test.tsx`.

15. Git y herramientas

Git

Concepto	Qué es
Repositorio	La carpeta con todo su historial (`.git`).
Commit	Una foto del proyecto con un mensaje.
Rama	Una línea de trabajo paralela.
Merge	Juntar los cambios de una rama en otra.
Remoto / push / pull	La copia en GitHub / subir / traer.
Pull request	Pedido para juntar una rama, con revisión.
`.gitignore`	Lo que no se guarda: `.env`, `node_modules`, `target`.

Conventional Commits (`tipo(alcance): descripción en español`): `feat` (funcionalidad), `fix` (corrección), `test`, `docs`, `refactor` (sin cambiar comportamiento) y `build`. Ejemplo: `feat(transferencia): descontar stock con criterio FEFO`. Commits chicos y coherentes.

Maven y npm

- **Maven** lee `pom.xml` (dependencias y plugins). El **wrapper** `mvnw` descarga la versión correcta. `./mvnw verify` = formato + unitarias + integración; `spotless:apply` formatea; `spring-boot:run` levanta el backend. El **BOM** de Spring Boot elige versiones compatibles.
- **npm** lee `package.json` (`dependencies`, `devDependencies`, `scripts`); `package-lock.json` fija las versiones exactas. Comandos: `install`, `run dev`, `run build`, `run lint`, `run typecheck`, `test`, `run api:tipos`, `run mock` y `run capturas`.

Calidad automática y documentación de la API

Herramienta	Qué controla
Spotless + google-java-format	Formato del Java (`verify` falla si no está aplicado).
ESLint	Errores comunes, reglas de React, accesibilidad, prohibiciones. Sin advertencias.
Prettier	Formato del frontend.
TypeScript estricto	Tipos.
knip	Archivos, exports y dependencias sin usar.

El objetivo: que el estilo y los errores obvios los controle una máquina, y la revisión humana se concentre en la lógica. **OpenAPI** describe todos los endpoints (`/v3/api-docs`, generado por springdoc) y **Swagger UI** permite probarlos desde el navegador (`/swagger-ui.html`, botón "Authorize" para el token).

IA: se usó un asistente de programación como herramienta. Cada parte se especificó con criterios de aceptación antes, se revisó y se verificó con pruebas; las decisiones quedaron registradas con su justificación.

16. Glosario rápido

Término	Significado
ACID	Atomicidad, consistencia, aislamiento y durabilidad de una transacción.
API / endpoint	Lo que un programa le ofrece a otro / un método HTTP + una ruta.
Baja lógica	Desactivar en lugar de borrar (`activo = false`).
BCrypt	Función de hash lenta y con sal para contraseñas.
Bean	Objeto que crea y administra Spring.
Bloqueo pesimista	`SELECT … FOR UPDATE`: reservar filas mientras se usan.
CHECK	Restricción de la base que una condición se cumple sí o sí.
CI	Integración continua: pruebas automáticas en cada push.
Clock	El reloj del que sale "hoy", con zona de Buenos Aires.
CORS	Regla del navegador sobre pedidos a otro origen.
CSP	Política que limita qué scripts ejecuta el navegador.
CSRF	Ataque que usa las cookies que el navegador manda solo.
Deadlock	Dos transacciones que se esperan mutuamente.
DTO	Objeto (record) solo con los datos que entran o salen por la API.
Entidad	Clase que representa una tabla (`@Entity`).
FEFO / FIFO	Primero vence, primero sale / primero entra, primero sale.
FK / PK	Clave foránea / clave primaria.
Flyway	Aplica las migraciones SQL numeradas.
Hash	Transformación irreversible de un texto.
Inyección de dependencias	Recibir las dependencias en el constructor en vez de crearlas.
JPA / Hibernate / ORM	Mapear clases a tablas / quien lo implementa / la técnica.
JWT	Token firmado que dice quién sos.
Migración	Archivo `.sql` numerado que cambia el esquema.
Mock	Objeto falso que reemplaza una dependencia en una prueba.
Normalización / 3FN	Organizar las tablas para no repetir datos.
ProblemDetail	Formato estándar de error (RFC 9457) con `codigo`.
Proxy	Intermediario que reenvía `/api` al backend.
Record	Clase de datos inmutable de Java.
SPA	Aplicación que se carga una vez y cambia sin recargar.
Stateless	Sin memoria entre pedidos.
Testcontainers	PostgreSQL real en Docker durante las pruebas.
Transacción	Grupo de operaciones que se hacen todas o ninguna.
Trazabilidad	Poder saber quién, cuándo y por qué cambió cada stock.
XSS	Inyectar código que se ejecute en el navegador de otro.

17. Preguntas para practicar

Respondé cada una **en voz alta** antes de leer la respuesta.

Lo general y la web

1. ¿Qué hace el sistema, en dos oraciones? Gestiona el stock por lote de la farmacia de un hospital público: registra ingresos y transferencias a las áreas aplicando FEFO, bajas y ajustes, con trazabilidad completa. Avisa vencimientos y faltantes y genera reportes, con acceso por roles.

2. ¿Dónde está la seguridad: en el frontend o en el backend? En el backend, porque el frontend corre en la máquina del usuario y se puede modificar. El frontend solo oculta lo que no corresponde, por comodidad.

3. ¿Qué diferencia hay entre 401 y 403? 401: no sé quién sos (sin token, inválido, vencido o usuario desactivado). 403: sé quién sos pero no tenés permiso.

4. ¿Qué devuelve el sistema cuando falta stock en una transferencia? 409 con `codigo: STOCK_INSUFICIENTE` y una lista `faltantes` con lo disponible y lo solicitado de cada medicamento. No se modifica nada.

5. ¿Por qué no hace falta configurar CORS? Porque el navegador le habla a un solo origen: Vite o nginx sirven el frontend y reenvían `/api` al backend.

Base de datos

6. ¿Por qué el lote no guarda el nombre del medicamento? Por la 3FN: el nombre depende de `medicamento_id`, no del lote. Si se repitiera, al corregirlo podrían quedar distintos. Se obtiene con un JOIN.

7. ¿Por qué el stock total no es una columna? Porque depende de los lotes y de la fecha de hoy (excluye los vencidos). Guardarlo arriesgaría que no coincida con la suma real. Se calcula con SUM y GROUP BY.

8. ¿Hay alguna redundancia en el modelo? `movimiento_stock.saldo_resultante`, un dato histórico de auditoría que nunca se actualiza, así que no genera anomalías.

9. ¿Para qué sirve `detalle_transferencia`? Resuelve la relación muchos a muchos entre transferencias y lotes, y registra cuántas unidades salieron de cada lote.

10. ¿Por qué no se edita una migración aplicada? Porque las bases existentes no recibirían el cambio y quedarían distintas; Flyway lo detecta y no arranca. Los cambios van en una migración nueva.

Backend

11. ¿Qué hace cada capa? Controller: traduce HTTP ↔ Java, valida formato y permiso, sin lógica. Service: reglas de negocio dentro de una transacción. Repository: consultas a la base.

12. ¿Por qué se usa un Clock y no `LocalDate.now()`? Por la zona horaria (el servidor corre en UTC) y por las pruebas (con un reloj fijo los bordes dan siempre igual).

13. ¿Por qué no se devuelven entidades? Si se devolviera `Usuario` saldría el hash de la contraseña, y la API quedaría atada a la base. Se devuelven DTO.

Reglas de negocio

14. Un lote vence hoy. ¿Está vencido? ¿Se puede transferir? No: es POR_VENCER y se puede transferir. Vencido es fecha anterior a hoy.

15. Hay un lote de 80 que vence en enero y uno de 200 que vence en diciembre. Piden 100. ¿Y si piden 250? Con 100: salen 100 del de diciembre. Con 250: 200 del de diciembre (queda en 0) y 50 del de enero (quedan 30), en dos renglones.

16. Se piden tres medicamentos y falta stock de uno. ¿Qué pasa con los otros dos? No se descuenta nada (todo o nada). Se responde 409 con todos los faltantes y el frontend conserva lo cargado.

17. ¿Cómo se resolvió la concurrencia? Con `SELECT … FOR UPDATE` en una sola consulta ordenada por id: la segunda transferencia espera y ve el stock actualizado, y el orden fijo evita el deadlock. Lo demuestra `TransferenciaConcurrenciaIT`.

18. ¿Cómo se corrige una transferencia equivocada? No se anula ni se edita: se hace un ajuste del lote con el motivo, que queda registrado.

19. ¿Se puede desactivar un medicamento con stock? ¿Y un proveedor? Medicamento con stock no vencido: no. Proveedor: sí; sus lotes siguen disponibles, pero no se lo elige en ingresos nuevos.

Seguridad

20. ¿Cómo se guardan las contraseñas? Como hash BCrypt con costo 12: irreversible, con sal y lento a propósito.

21. ¿Por qué el JWT no lleva el rol? Porque los permisos se leen de la base en cada pedido: una baja o un cambio de rol tiene efecto inmediato.

22. Si alguien cambia el id dentro del token, ¿qué pasa? La firma deja de coincidir (no conoce la clave) y el servidor responde 401.

23. ¿Por qué CSRF está deshabilitado? Porque la API no usa cookies: el token va en `Authorization`, que el navegador no agrega solo.

24. ¿Por qué el login responde igual si el usuario no existe? Para no revelar qué usuarios existen; hasta el tiempo de respuesta es parecido.

Frontend y herramientas

25. ¿Para qué se usa TanStack Query? Para pedir datos con caché, estados de carga y error, reintento controlado e invalidación después de cada cambio.

26. ¿Por qué hay un solo archivo con `fetch`? Para que el token, el tiempo máximo y los errores se manejen igual en toda la app, y probarlo por separado.

27. ¿Por qué las fechas no pasan por `new Date()`? Porque `new Date('2026-11-30')` es medianoche UTC y en Argentina mostraría el 29.

28. ¿Qué pasa con una transferencia a medio cargar si vence la sesión? Se conserva: el borrador está en `sessionStorage` y se recupera al volver a ingresar en la misma pestaña.

29. ¿Dónde están los secretos? En variables de entorno (`.env`, que no se sube a git). En el código los tendría cualquiera con acceso al repositorio.

30. ¿Por qué PostgreSQL real en las pruebas y no H2? Para probar `CHECK`, `FOR UPDATE` y SQL específico de PostgreSQL. Con otra base, una prueba podría pasar y fallar en producción.

18. Checklist antes de la defensa

- [ ] Puedo explicar el sistema en un minuto.
- [ ] Puedo dibujar la arquitectura (navegador → frontend → backend → base).
- [ ] Puedo dibujar el diagrama ER y justificar la 3FN.
- [ ] Puedo explicar FEFO con un ejemplo numérico.
- [ ] Puedo explicar la concurrencia, el bloqueo y el deadlock.
- [ ] Puedo explicar login, BCrypt, JWT, roles, 401 y 403.
- [ ] Sé qué prueba demuestra cada problema del prototipo resuelto.
- [ ] Levanté el sistema con Docker y practiqué la demo.
- [ ] Leí el guion de la defensa.